

CẤU HÌNH DỰ ÁN & HƯỚNG DẪN TRIỂN KHAI / PROJECT SETTINGS & DEPLOYMENT GUIDE

Tài liệu này cung cấp hướng dẫn chi tiết từng bước dành cho nhà phát triển hoặc kỹ sư DevOps để thiết lập, cài đặt môi trường và triển khai hệ thống Trợ Lý AI Y Học Cổ Truyền (YHCT) ở cả môi trường phát triển (Development) và môi trường thực tế (Production).

This document provides a comprehensive, step-by-step guide for developers and DevOps engineers to configure, set up, and deploy the Traditional Vietnamese Medicine (TVM) AI Assistant system in both development and production environments.

MỤC LỤC / TABLE OF CONTENTS

TIẾNG VIỆT (VIETNAMESE VERSION)

1. Yêu cầu Hệ thống & Môi trường
2. Hướng dẫn Thiết lập Từng bước (Step-by-step Setup)
 - 2.1. Cơ sở dữ liệu đồ thị Neo4j
 - 2.2. Thiết lập FastAPI Backend
 - 2.3. Thiết lập React/Vite Frontend
 - 2.4. Đồ dữ liệu y văn vào Đồ thị (Data Ingestion)
3. Giải thích Chi tiết File Cấu hình .env
4. Triển khai Production (Production Deployment)
5. Khắc phục Sự cố Thường gặp (Troubleshooting)

ENGLISH (ENGLISH VERSION)

1. System Prerequisites
2. Step-by-Step Deployment Guide
 - 2.1. Neo4j Graph Database Setup
 - 2.2. FastAPI Backend Setup
 - 2.3. React/Vite Frontend Setup
 - 2.4. Loading Database (Data Ingestion)
3. Environment Variable Details (.env)
4. Production Deployment
5. Troubleshooting & FAQs

TIẾNG VIỆT (VIETNAMESE VERSION)

1. Yêu cầu Hệ thống & Môi trường Trước khi bắt đầu cài đặt, đảm bảo thiết bị hoặc máy chủ (VPS/Server) đáp ứng các điều kiện tối thiểu sau: Hệ điều hành: Windows 10/11, macOS, hoặc Linux (Ubuntu 20.04 LTS trở lên). Python: Phiên bản Python 3.10 hoặc 3.11 (Không khuyến nghị 3.12+ do một số thư viện học máy chưa tương thích hoàn toàn). Node.js: Phiên bản Node.js 18.x hoặc 20.x (kèm theo NPM). Ollama: Được cài đặt và chạy ngầm (cổng 11434) nếu muốn sử dụng mô hình embedding nội bộ hoặc mô hình LLM chạy local (như Qwen). Cơ sở dữ liệu: Neo4j AuraDB (Khuyến dùng gói đám mây Free/Professional) hoặc Neo4j Enterprise/Community Server chạy local. SQLite (Tích hợp sẵn trong Python, không cần cài đặt thêm).
2. Hướng dẫn Thiết lập Từng bước (Step-by-step Setup)
 - 2.1. Cơ sở dữ liệu đồ thị Neo4j

1. Đăng ký tài khoản trên Neo4j Aura (neo4j.com/cloud/platform/auradb/) và tạo một Instance cơ sở dữ liệu trống.
2. Tải xuống tệp thông tin kết nối chứa: URI, Username, và Password.
3. Lưu thông tin này để khai báo vào file .env ở bước sau.

2.2. Thiết lập FastAPI Backend Mở Terminal tại thư mục gốc của dự án và chạy các lệnh sau:

1. Khởi tạo môi trường ảo Python (Virtual Environment): Windows python -m venv venv
.\venv\Scripts\Activate.ps1 (Nếu dùng PowerShell) hoặc: .\venv\Scripts\activate.bat (Nếu dùng CMD)

Linux / macOS python3 -m venv venv source venv/bin/activate

2. Cài đặt các gói thư viện phụ thuộc: python -m pip install --upgrade pip pip install -r requirements.txt pip install PyMuPDF
3. Tạo tệp cấu hình môi trường .env: Sao chép file mẫu: cp .env.example .env (hoặc tạo file mới tên .env tại thư mục gốc). Điền đầy đủ các khóa kết nối (xem phần giải thích chi tiết ở mục 3).
4. Khởi chạy Backend ở môi trường phát triển: Chạy lệnh uvicorn trực tiếp uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

Hoặc sử dụng tệp khởi chạy có sẵn python run_api.py

2.3. Thiết lập React/Vite Frontend Mở cửa sổ Terminal mới, chuyển vào thư mục frontend:

1. Cài đặt các gói thư viện JavaScript: cd frontend npm install
2. Cấu hình API Endpoint: Tệp cấu hình .env của frontend được đặt sẵn tại frontend/.env. Đảm bảo trường VITE_API_URL trỏ đúng về địa chỉ Backend (mặc định: http://localhost:8000).
3. Khởi chạy Frontend ở chế độ phát triển (Development): npm run dev Giao diện người dùng mặc định khả dụng tại địa chỉ: http://localhost:5173

2.4. Đồ dữ liệu y văn vào Đồ thị (Data Ingestion) Sau khi đã thiết lập xong biến môi trường kết nối Neo4j trong .env, bạn tiến hành chạy scripts để tự động làm sạch và đồ dữ liệu tri thức vào Graph:

Đảm bảo bạn đang ở thư mục gốc của dự án và môi trường venv đã được kích hoạt python ingestion/deploy_to_neo4j_server.py

Script này sẽ đọc các tệp y văn đã được cấu trúc hóa trong thư mục storage/gold/ và khởi tạo toàn bộ thực thể (Vị thuốc, Bài thuốc, Tính vị, Quy kinh) cùng các mối quan hệ liên kết lên Neo4j Cloud.

3. Giải thích Chi tiết File Cấu hình .env

File .env nằm tại thư mục gốc quản lý tất cả cài đặt hệ thống. Dưới đây là ý nghĩa chi tiết:

Nhóm Biến: Cơ sở dữ liệu Tên Biến Môi Trường: NEO4J_URI Mô tả Chi tiết: Địa chỉ kết nối Neo4j (dùng bolt/neo4j kèm mã hóa ssl) Ví dụ Mẫu / Mặc định: neo4j+ssc://xxxxxx.databases.neo4j.io

Tên Biến Môi Trường: NEO4J_USER Mô tả Chi tiết: Tên tài khoản đăng nhập Neo4j Ví dụ Mẫu / Mặc định: neo4j

Tên Biến Môi Trường: NEO4J_PWD Mô tả Chi tiết: Mật khẩu truy cập cơ sở dữ liệu Neo4j Ví dụ Mẫu / Mặc định: Mật khẩu do Aura cấp

Tên Biến Môi Trường: NEO4J_DB_NAME Mô tả Chi tiết: Tên cơ sở dữ liệu Neo4j Ví dụ Mẫu / Mặc định: neo4j

Tên Biến Môi Trường: SQLITE_DB_PATH Mô tả Chi tiết: Tên/đường dẫn tệp SQLite lưu user & giao dịch Ví dụ Mẫu / Mặc định: yhct_database.db

Nhóm Biến: Trí tuệ Nhân tạo Tên Biến Môi Trường: GEMINI_API_KEY Mô tả Chi tiết: Khóa API chính kết nối Google Gemini Ví dụ Mẫu / Mặc định: AIzaSyD-xxx

Tên Biến Môi Trường: GEMINI_FALLBACK_KEYS Mô tả Chi tiết: Danh sách khóa dự phòng Gemini (phân cách bằng dấu phẩy) Ví dụ Mẫu / Mặc định: Key_Phụ_1,Key_Phụ_2

Tên Biến Môi Trường: OPENAI_API_KEY Mô tả Chi tiết: Khóa API OpenAI (tùy chọn so sánh/dự phòng) Ví dụ Mẫu / Mặc định: sk-proj-xxx

Tên Biến Môi Trường: QWEN_API_URL Mô tả Chi tiết: URL kết nối máy chủ Ollama hoặc Qwen chạy cục bộ Ví dụ Mẫu / Mặc định: http://localhost:11434

Tên Biến Môi Trường: GOOGLE_APPLICATION_CREDENTIALS Mô tả Chi tiết: Đường dẫn đến tệp Service Account JSON (cho OCR Document AI) Ví dụ Mẫu / Mặc định: config/yhct-knowledge-graph.json

Nhóm Biến: Xác thực Bảo mật Tên Biến Môi Trường: JWT_SECRET_KEY Mô tả Chi tiết: Khóa bảo mật để ký mã JWT xác thực đăng nhập Ví dụ Mẫu / Mặc định: Chuỗi ngẫu nhiên có độ dài lớn hơn 32 ký tự

Tên Biến Môi Trường: ALGORITHM Mô tả Chi tiết: Thuật toán băm mã khóa JWT Ví dụ Mẫu / Mặc định: HS256

Tên Biến Môi Trường: ACCESS_TOKEN_EXPIRE_MINUTES Mô tả Chi tiết: Thời gian hết hạn của token đăng nhập (phút) Ví dụ Mẫu / Mặc định: 1440 (Tương đương 24 giờ)

Nhóm Biến: Đăng nhập Google Tên Biến Môi Trường: GOOGLE_CLIENT_ID Mô tả Chi tiết: Client ID đăng ký trên Google Cloud Console Ví dụ Mẫu / Mặc định: xxxx-xxx.apps.googleusercontent.com

Tên Biến Môi Trường: GOOGLE_CLIENT_SECRET Mô tả Chi tiết: Client Secret đi kèm Client ID Ví dụ Mẫu / Mặc định: GOCSPX-xxxx

Tên Biến Môi Trường: GOOGLE_REDIRECT_URI Mô tả Chi tiết: URL Backend tiếp nhận mã đăng nhập từ Google Ví dụ Mẫu / Mặc định: http://localhost:8000/auth/google/callback

Tên Biến Môi Trường: FRONTEND_URL Mô tả Chi tiết: Địa chỉ client chuyển hướng sau khi login thành công Ví dụ Mẫu / Mặc định: http://localhost:5173

Nhóm Biến: Fintech & SePay Tên Biến Môi Trường: SEPAY_API_KEY Mô tả Chi tiết: Token API kết nối tài khoản SePay đối soát Ví dụ Mẫu / Mặc định: sepay_api_token_here

Tên Biến Môi Trường: SECRET_XOR_KEY Mô tả Chi tiết: Khóa XOR giải mã ID giao dịch ngân hàng Ví dụ Mẫu / Mặc định: 387835

Tên Biến Môi Trường: NAME_WEB Mô tả Chi tiết: Định danh tiền tố viết tắt cho nội dung chuyển khoản Ví dụ Mẫu / Mặc định: YHCT (Dạng YHCTNAPTOKEN HEX)

4. Triển khai Production (Production Deployment)

Khi đưa ứng dụng lên máy chủ thực tế (VPS/Server Linux), hãy tuân thủ các bước khuyến nghị sau:

Step 1: Triển khai Frontend (React)

1. Di chuyển vào thư mục frontend và xây dựng bản tối ưu hóa cho Production: `cd frontend`
`npm run build`
2. Nội dung trang web tĩnh sẽ được tạo ra trong thư mục frontend/dist.
3. Cấu hình máy chủ Nginx để phục vụ thư mục tĩnh này. Mẫu cấu hình Nginx tham khảo:
`server { listen 80; server_name yourdomain.com;`

```
location / { root /var/www/yhct-frontend/dist; try_files $uri $uri/ /index.html; }
```

```
location /api/ { proxy_pass http://127.0.0.1:8000/; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; } }
```

Step 2: Triển khai Backend (FastAPI)

1. Thay vì chạy bằng lệnh `uvicorn` phát triển, hãy cài đặt Gunicorn làm Process Manager để đảm bảo hệ thống chịu tải tốt: `pip install gunicorn uvicorn`
2. Khởi chạy dịch vụ chạy ngầm thông qua Systemd Service trên Linux: Tạo tệp `/etc/systemd/system/yhct-backend.service`:
`[Unit] Description=YHCT FastAPI Backend Service After=network.target`

```
[Service] User=ubuntu WorkingDirectory=/var/www/yhct-backend ExecStart=/var/www/yhct-backend/venv/bin/gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker -b 127.0.0.1:8000 Restart=always
```

```
[Install] WantedBy=multi-user.target
```

3. Kích hoạt và khởi chạy service: `sudo systemctl daemon-reload` `sudo systemctl enable yhct-backend` `sudo systemctl start yhct-backend`
4. Khắc phục Sự cố Thường gặp (Troubleshooting)

Lỗi "Neo4j connection timeout": Nguyên nhân: Do sai địa chỉ URI hoặc mật khẩu trong `.env`. Khắc phục: Đảm bảo URI kết nối AuraDB có hậu tố `+ssc` (ví dụ: `neo4j+ssc://...`) để bật chế độ bảo mật SSL tự động.

Lỗi OAuth 2.0 Đăng nhập thất bại (Redirect URI Mismatch): Nguyên nhân: URL redirect khai báo trên Google Cloud Console không trùng khớp hoàn toàn với biến `GOOGLE_REDIRECT_URI` trong `.env`. Khắc phục: Truy cập Google Cloud Console -> Credentials -> OAuth 2.0 Client -> Authorized redirect URIs và bổ sung chính xác địa chỉ redirect của bạn (Ví dụ: <https://yourdomain.com/auth/google/callback>).

SePay Webhook không hoạt động: Nguyên nhân: SePay không thể gửi Webhook tới IP/domain cục bộ (localhost). Khắc phục: Trên môi trường phát triển (Local), cần sử dụng công cụ ngrok hoặc cloudflare tunnel để ánh xạ cổng 8000 ra internet công khai, sau đó khai báo URL công khai này vào cấu hình Webhook của SePay.

ENGLISH (ENGLISH VERSION)

1. System Prerequisites Before initializing the environment, ensure the system or remote server meets the following requirements: Operating System: Windows 10/11, macOS, or Linux (Ubuntu 20.04 LTS or newer). Python: Version Python 3.10 or 3.11 (Avoid 3.12+ due to compatibility issues with certain machine learning dependencies). Node.js: Version Node.js 18.x or 20.x (comes with NPM). Ollama: Installed and running locally (default port 11434) if you intend to run local embedding models (such as bge-m3) or local LLMs (like Qwen). Databases: Neo4j AuraDB (Free or Professional cloud instances recommended) or a local Neo4j Community/Enterprise Server. SQLite (Embedded within Python, no separate installation required).

2. Step-by-Step Deployment Guide

2.1. Neo4j Graph Database Setup

1. Register an account on Neo4j Aura (neo4j.com/cloud/platform/auradb/) and spawn a new blank database instance.
2. Download the generated credentials text file containing the URI, Username, and Password.
3. Keep this connection payload secure; it will be declared in the `.env` file during the backend setup.

2.2. FastAPI Backend Setup Open a Terminal in the project root directory and run the following:

1. Initialize the Python Virtual Environment: Windows `python -m venv venv` `.\venv\Scripts\Activate.ps1` (For PowerShell) or: `.\venv\Scripts\activate.bat` (For CMD)

Linux / macOS `python3 -m venv venv` `source venv/bin/activate`

2. Install Library Dependencies: `python -m pip install --upgrade pip` `pip install -r requirements.txt` `pip install PyMuPDF`
3. Configure Environment Variables: Clone the example file: `cp .env.example .env` (or create a new `.env` file in the root folder). Declare all key connections (Refer to Section 3 for variable explanations).
4. Launch the Backend in Development Mode: Direct launch `uvicorn app.main:app --reload --host 0.0.0.0 --port 8000`

Or use the helper script `python run_api.py`

2.3. React/Vite Frontend Setup Open a separate Terminal window and step into the frontend folder:

1. Install JavaScript Packages: `cd frontend npm install`
2. Point API Endpoint: The frontend configuration is managed in `frontend/.env`. Verify that `VITE_API_URL` points correctly to the Backend URL (defaults to `http://localhost:8000`).
3. Launch Frontend in Development Mode: `npm run dev` The UI dashboard is accessible at: `http://localhost:5173`

2.4. Loading Database (Data Ingestion) Once your database credentials are set up inside the `.env` file, run the ingestion sequence to populate the graph:

Run this from the root directory with `venv` active `python ingestion/deploy_to_neo4j_server.py`

This command parses the formatted TVM data files under `storage/gold/` and automatically populates all nodes (Herbs, Remedies, Properties) and their relationships to your Neo4j database.

3. Environment Variable Details (`.env`)

The `.env` file in the root directory manages system-wide configurations. Here is a breakdown:

Category: Database Variable Key: `NEO4J_URI` Description: Neo4j connection URL (bolt/neo4j secure format) Example / Default Value: `neo4j+ssc://xxxxxx.databases.neo4j.io`

Variable Key: `NEO4J_USER` Description: Username to log into Neo4j Example / Default Value: `neo4j`

Variable Key: `NEO4J_PWD` Description: Password to log into Neo4j Example / Default Value: Instance password

Variable Key: `NEO4J_DB_NAME` Description: Database identifier Example / Default Value: `neo4j`

Variable Key: `SQLITE_DB_PATH` Description: File path to SQLite database for users and payments Example / Default Value: `yhct_database.db`

Category: Artificial Intelligence Variable Key: `GEMINI_API_KEY` Description: Primary API Key to connect to Google Gemini Example / Default Value: `AIzaSyD-xxx`

Variable Key: `GEMINI_FALLBACK_KEYS` Description: Comma-separated list of fallback Gemini API keys Example / Default Value: `Key_2,Key_3`

Variable Key: `OPENAI_API_KEY` Description: OpenAI API Key (optional fallback / benchmarking) Example / Default Value: `sk-proj-xxx`

Variable Key: `QWEN_API_URL` Description: Connection URL for local Ollama / Qwen model service Example / Default Value: `http://localhost:11434`

Variable Key: GOOGLE_APPLICATION_CREDENTIALS Description: Google Service Account JSON path (for OCR Document AI) Example / Default Value: config/yhct-knowledge-graph.json

Category: Security / JWT Variable Key: JWT_SECRET_KEY Description: Cryptographic secret to sign user login tokens Example / Default Value: Random secure string (greater than 32 characters)

Variable Key: ALGORITHM Description: Cryptographic hashing algorithm for JWT Example / Default Value: HS256

Variable Key: ACCESS_TOKEN_EXPIRE_MINUTES Description: Token session duration in minutes Example / Default Value: 1440 (Equivalent to 24 hours)

Category: Google Authentication Variable Key: GOOGLE_CLIENT_ID Description: Google OAuth 2.0 Client ID Example / Default Value: xxxx-xxx.apps.googleusercontent.com

Variable Key: GOOGLE_CLIENT_SECRET Description: Google OAuth 2.0 Client Secret Example / Default Value: GOCSPX-xxxx

Variable Key: GOOGLE_REDIRECT_URI Description: Backend callback route for Google authentication Example / Default Value: http://localhost:8000/auth/google/callback

Variable Key: FRONTEND_URL Description: Redirect target client URL after successful login Example / Default Value: http://localhost:5173

Category: Fintech / Payment Variable Key: SEPAY_API_KEY Description: API Access Token for SePay automated reconciliations Example / Default Value: sepay_api_token_here

Variable Key: SECRET_XOR_KEY Description: Cryptographic XOR key for encoding transaction codes Example / Default Value: 387835

Variable Key: NAME_WEB Description: Web site uppercase prefix code for bank transfers Example / Default Value: YHCT (Generates YHCTNAPTOKEN HEX)

4. Production Deployment

To host this application permanently on a remote production server (e.g., VPS / Linux Server), follow these steps:

Step 1: Deploy Frontend (React)

1. Navigate to the frontend folder and run the production build: `cd frontend npm run build`
2. The compiled static code is generated in `frontend/dist`.
3. Configure Nginx to serve this directory. Reference Nginx Server Block: `server { listen 80; server_name yourdomain.com;`

```
location / { root /var/www/yhct-frontend/dist; try_files $uri $uri/ /index.html; }
```

```
location /api/ { proxy_pass http://127.0.0.1:8000/; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; } }
```

Step 2: Deploy Backend (FastAPI)

1. Install Gunicorn process manager to handle concurrent requests under load: `pip install gunicorn uvicorn`
2. Configure a Systemd Service to run Gunicorn in the background on Linux: Create file `/etc/systemd/system/yhct-backend.service`:
`[Unit] Description=YHCT FastAPI Backend Service After=network.target`

`[Service] User=ubuntu WorkingDirectory=/var/www/yhct-backend ExecStart=/var/www/yhct-backend/venv/bin/gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker -b 127.0.0.1:8000 Restart=always`

`[Install] WantedBy=multi-user.target`

3. Reload daemon and start the service: `sudo systemctl daemon-reload sudo systemctl enable yhct-backend sudo systemctl start yhct-backend`
4. Troubleshooting & FAQs

"Neo4j connection timeout": Cause: Incorrect connection URI or credentials in the `.env` file. Solution: Double check that your AuraDB connection URI contains the `+ssc` suffix (e.g., `neo4j+ssc://...`) to enforce TLS encryption.

Google Login fails with "Redirect URI Mismatch": Cause: The redirect URI configured on your Google Cloud Console project does not match the `GOOGLE_REDIRECT_URI` variable. Solution: Go to Google Cloud Console -> Credentials -> OAuth 2.0 Client -> Authorized redirect URIs and verify the address (e.g., <https://yourdomain.com/auth/google/callback>).

SePay Webhooks are not arriving: Cause: SePay cannot connect to localhost or internal private IP addresses. Solution: For local development, use tools like ngrok or Cloudflare Tunnels to expose port 8000 to the internet, and configure this public domain endpoint inside the SePay webhook settings.